CaptureForge User Manual

A general-purpose hybridization-capture bait (probe) design pipeline — v0.1.0

Bright Adu
Genomics and Bioinformatics Core Facility, Noguchi Memorial Institute for Medical Research, University of Ghana, Legon, Ghana

13 June 2026

Contents

1 Introduction **4**
- 1.1 When to use hybridization capture (vs amplicon vs WGS) . . . 4
- 1.2 Who this manual is for . . . 4

2 Concepts **4**
- 2.1 What a capture bait (probe) is . . . 4
- 2.2 How hybridization capture works . . . 4
- 2.3 Per-region design classes . . . 5

3 Pipeline architecture **5**

4 Installation **6**
- 4.1 Prerequisites . . . 6
- 4.2 Local install (laptop or workstation — no HPC required) . . . 6
  - 4.2.1 Option A — conda / mamba environment . . . 6
  - 4.2.2 Option B — Docker (avoids the Apple-Silicon PATH issue entirely) . . . 8
- 4.3 Shared HPC install (this cluster as the worked example) . . . 8

5 Preparing inputs **9**
- 5.1 On-ramp 1 — Plain-text symbols (zero friction) . . . 9
- 5.2 On-ramp 2 — CSV/TSV spec (the reusable format) . . . 9
  - 5.2.1 Scaffolding a spec from bare symbols . . . 9
- 5.3 On-ramp 3 — Rich curated workbook (`.xlsx`) . . . 10
  - 5.3.1 How `Variant class` text maps to design classes . . . 10
  - 5.3.2 Aliases and anchors . . . 10
- 5.4 Worked tutorial — starting from a gene name (p53 → TP53) . . . 10
  - 5.4.1 Step 1 — get the symbol right (the most common trap) . . . 10
  - 5.4.2 Step 2 — choose an on-ramp . . . 10
  - 5.4.3 Step 3 — make sure references exist . . . 11
  - 5.4.4 Step 4 — run . . . 11
  - 5.4.5 Step 5 — read the outputs . . . 12
  - 5.4.6 Honest caveat — one small gene is not a pool . . . 12

6 Reference data **12**
- 6.1 Fetching human GRCh38 references . . . 12
- 6.2 The no-alt reference invariant (read this) . . . 12
- 6.3 Building the specificity index . . . 13

7 Configuration **13**
- 7.1 Profiles . . . 13
- 7.2 Key parameters (real defaults from `nextflow.config`) . . . 13

8 Running the pipeline **14**
- 8.1 Quickstart — the `test` profile (10 genes, ~2 min) . . . 14
- 8.2 Full run — `local` . . . 14
- 8.3 Full run — `hpc_slurm` with Apptainer . . . 15

9 Running on different organisms **15**
- 9.1 Human (GRCh38) — the baseline . . . 15
- 9.2 Mouse (GRCm39, Ensembl) — MANE-absent fallback . . . 16
- 9.3 Plasmodium falciparum — non-model, AT-rich genome . . . 16
- 9.4 “Any organism” checklist . . . 17

10 Outputs explained **17**

10.1 order_package/ . . . 17
10.2 stage9_qc/ . . . 18
10.3 qc_dashboard.html and figures/ . . . 18

11 Interpreting QC and coverage **18**
11.1 Mappable vs raw coverage . . . 18
11.2 The three-tier per-gene honesty . . . 18
11.3 CNV read-depth callability is the decision metric — not raw CNV coverage . . . 20
11.4 Reading a per-gene track plot — real vs uncapturable gaps . . . 20
11.5 Composition QC — GC and Tm . . . 20
11.6 Specificity QC — the off-target distribution . . . 20
11.7 Boosting / dedup — copy number . . . 20
11.8 Re-tiling convergence — when non-convergence is correct . . . 20

12 Cost and ordering **25**
12.1 Pool cost and tiers . . . 25

13 Troubleshooting **25**
13.1 Known doc-vs-behaviour notes . . . 27

14 From design to order **27**
14.1 Choosing a bait format (bait_format) . . . 27
14.2 Mapping to a synthesizer template . . . 28
14.3 Pre-order checklist . . . 28

15 Extending CaptureForge **28**

16 Reproducibility and provenance **28**

17 Glossary **29**

18 FAQ **29**

19 Citation **29**

20 Version / changelog **29**

1 Introduction

CaptureForge turns a **gene list plus per-gene design intent** into **order-ready biotinylated-bait oligo sequences** and a full QC/design report. It is a reusable, **disease- and organism-agnostic** method tool: point it at any gene list (any disease, any study) and, where feasible, any reference genome, by changing **inputs and configuration only**, with no code edits.

Everything upstream of physical oligo synthesis is in scope. Synthesis is outsourced, so the final deliverables are **order-ready sequence files**, a self-contained QC dashboard, and a clean `final_covered_targets.bed` handoff for a downstream variant-analysis pipeline (a separate tool).

> CaptureForge **designs** baits. It does not order, synthesize, or contact any vendor, and it does not run downstream variant analysis.

1.1 When to use hybridization capture (vs amplicon vs WGS)

Approach	Best when	Trade-offs
Hybridization capture (CaptureForge)	Tens to thousands of targets; large or scattered regions (whole genes, promoters, CNV loci); flexible panel that you will revise	Higher up-front design effort; even coverage; tolerant of variation under the probe
Amplicon / multiplex PCR	Small, fixed sets of short, well-defined regions (hotspots); lowest cost per sample at small scale	Primer design is brittle near variants/repeats; hard to scale to whole genes
Whole-genome sequencing	You need the whole genome, or targets are unknown	Most sequencing per sample; most cost; most data to store/analyze

Use capture when you want **gene-level** coverage of a curated panel with room to grow, and you can tolerate masked/uncapturable sequence (repeats, segmental duplications) being intentionally left out.

1.2 Who this manual is for

Bioinformaticians and core-facility staff who will **run** CaptureForge to design a panel, **interpret** its QC, and **prepare an order**. It assumes comfort with the command line, Nextflow, and conda/containers, but explains the genomics concepts where they matter for decisions.

---

2 Concepts

2.1 What a capture bait (probe) is

A **bait** (or **probe**) is a short, single-stranded, biotinylated oligonucleotide — in CaptureForge, **120 nt** by default — whose sequence is complementary to a region you want to sequence. A **panel** is the **set of baits** that together tile all your targets.

2.2 How hybridization capture works

1. **Library prep** — fragment sample DNA and add sequencing adapters.
2. **Hybridization** — denature the library and add the bait pool; baits anneal to their complementary fragments in solution.

3. **Streptavidin pulldown** — magnetic streptavidin beads bind the biotin on the baits, pulling down the bait–fragment duplexes.
4. **Wash** — wash away unbound (off-target) fragments.
5. **Enriched library** — elute, amplify, and sequence — now strongly enriched for your targets.

CaptureForge designs step 2's reagent: the bait pool. A bait tolerates some mismatch to its target (so it captures variants *under* the probe), which is also why **specificity** matters — a bait too similar to an off-target locus will pull that locus down too.

2.3 Per-region design classes

Not every target should be tiled the same way. CaptureForge tags each target interval with a **design class** that drives its tiling density, masking policy, GC policy, and boosting:

Class	What it targets	Tiling / policy
`coding`	canonical-transcript exons (incl. UTRs) ± splice flanks	2× tiling, step = ½ bait; standard masking
`promoter`	strand-aware TSS window	3× tiling, step = ⅓ bait; low-complexity **relaxed**; high-GC bound relaxed; boosted
`regulatory`	Regulatory-Build / cCRE features over the locus (if provided)	as promoter
`cnv`	the whole gene locus	2× **even** tiling for read-depth CNV calling
`str`	the **unique flanks** of a short-tandem-repeat locus	the repeat core is removed at masking; tile flanks only
`anchor`	an rsID ± flank	ensure ≥2 baits span the variant; minimal masking

`coding` is always implied for every gene. The other classes are added per your input (see *Preparing inputs*). This is why a CNV gene and an STR gene that overlap the same locus produce visibly different bait layouts.

---

3 Pipeline architecture

CaptureForge is a **Nextflow (DSL2)** pipeline. `main.nf` calls one subworkflow (`subworkflows/captureforge.nf`) that chains 14 processes: a FASTA-index step, the ten design stages, and two visualization steps. Each stage is a module in `modules/` that wraps a tool in `bin/`, writes its outputs under `results/<outdir>/stageN_*/`, and emits a `stageN_summary.json` of metrics.

Stage	Module / tool	Consumes	Emits
1 — Target definition	`define_targets.py`	gene list + GTF	`targets.raw.bed` (labelled `gene|class|subtype`)
2 — Masking	`apply_masking.py` (dustmasker + rmsk/simpleRepeat)	`targets.raw.bed` + genome	`targets.masked.bed`, `targets.lost_to_mask.bed`, `*.str_core.bed`
3 — Tiling	`tile_candidates.py` (bedtools)	masked targets + genome	`candidates.bed`, `candidates.fasta`

Stage	Module / tool	Consumes	Emits
4 — Specificity	`specificity_filter.py` (blastn / bowtie2)	candidates + specificity index	`candidates.specpass.bed`, `candidates.spec.tsv`
5 — Composition	`composition_filter.py` (primer3)	spec-pass baits	`composition.pass.bed/fasta`, `composition.tsv`
6 — Re-tiling	`gap_retile.py` (reuses 4+5)	masked targets + passing baits	`retiled.pass.bed/fasta`, `residual_gaps.bed`
7 — Boost / dedup	`boost_dedup.py`	re-tiled baits	`baits.dedup.bed/fasta/tsv`
8 — Synthesis	`scaffold_synthesis.py`	dedup baits	`synthesis.fasta`, `synthesis.tsv`
9 — QC aggregation	`aggregate_qc.py`	all stage summaries + BEDs	`final_covered_targets.bed`, `metrics.json`, `coverage_by_{gene,class}.tsv`
10 — Export	`export_package.py`	dedup + synthesis + QC	`baits.fasta`, `baits.csv`, `order_generic.tsv`, `MANIFEST`, `provenance.json`
viz	`make_figures.py`, `make_dashboard.py`	QC + BEDs	`figures/*.png/svg`, `qc_dashboard.html`

The pipeline is driven by **profiles** (`-profile test|local|hpc_slurm`) that select the executor and resources, composed with a **packaging** profile (`conda|docker|apptainer` = how dependencies are provided). The comma composes them: an **execution** profile (`local` = this machine — any Linux/macOS, Windows via WSL2; `hpc_slurm` = a cluster) plus a packaging engine, e.g. `-profile local,conda`. An interactive DAG of the run is also written to `results/<outdir>/pipeline_info/dag.html`, alongside `report.html`, `timeline.html`, and `trace.txt`.

---

4 Installation

4.1 Prerequisites

- **Nextflow** ≥ 23.10 (needs **Java 17+**). The repo also bundles a launcher at `./.tools/nextflow`.
- **conda / mamba** (for the pinned environment) **or** Docker / Apptainer (for the container).
- The **`captureforge` command**: `pip install -e .` from the repo root registers it on PATH (a thin front door — `captureforge run/panel/index`). The raw forms still work; `captureforge` is recommended.

CaptureForge supports two audiences: **(1) local users** on a personal laptop or workstation with no cluster, and **(2) a shared HPC** install used by many people. Both are covered below.

4.2 Local install (laptop or workstation — no HPC required)

You can install and run CaptureForge **entirely on a personal computer**. The `test` profile and small panels run comfortably on a laptop; a full whole-genome–specificity run is heavier but still completes on a workstation (it takes longer — see *Running the pipeline*). No cluster, scheduler, or admin rights are needed.

4.2.1 Option A — conda / mamba environment

The pinned environment in `env/captureforge.yml` provides the whole toolchain (bedtools, samtools, seqkit, bowtie2, bwa-mem2, genmap, BLAST/dustmasker, primer3, pysam, Biopython, pyfaidx, pandas, openpyxl, jinja2, matplotlib, ViennaRNA, …).

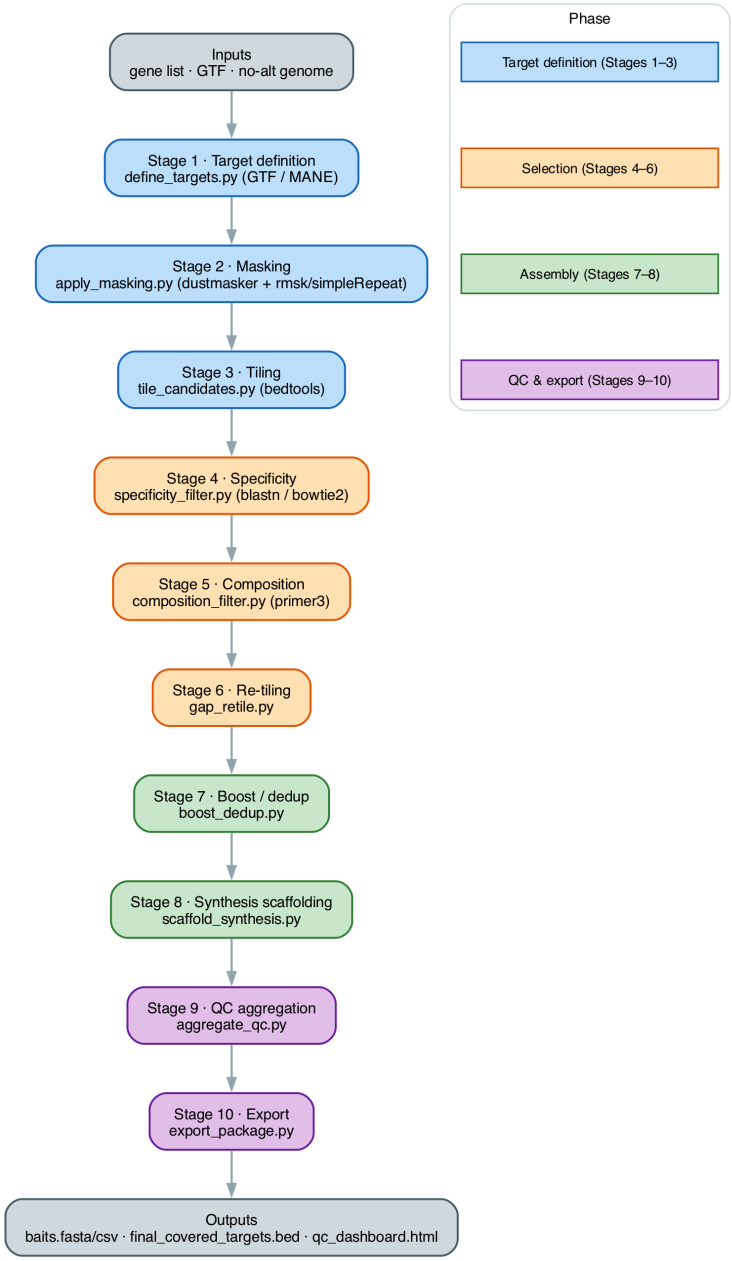


Figure 1: The 10-stage CaptureForge flow, color-coded by phase: target definition (blue) → selection (orange) → assembly (green) → QC & export (purple); grey nodes are inputs/outputs. Each node names the stage and its primary tool. Follow the arrows top-to-bottom; the same flow is what `main.nf` wires.

```
# Linux (native):
mamba env create -f env/captureforge.yml

# macOS Apple Silicon (arm64): build under Rosetta so all bioconda packages resolve
CONDA_SUBDIR=osx-64 mamba env create -f env/captureforge.yml

# activate the environment so `captureforge` installs into it, not your base environment
conda activate captureforge

# register the `captureforge` command (into the activated env)
pip install -e .
captureforge --version    # success: prints e.g. "captureforge 0.1.0"
```

> **Apple-Silicon PATH note.** The env builds as osx-64 under Rosetta. If you run `-profile <x>,conda`, Nextflow manages the environment for you. If instead you pre-activate the env and run without the `conda` engine, make sure the env's `bin` is first on PATH, so `/usr/bin/env python3` resolves to the env's interpreter (not Homebrew's):
>
> ```
> conda activate captureforge && export PATH="$CONDA_PREFIX/bin:$PATH"
> ```

4.2.2 Option B — Docker (avoids the Apple-Silicon PATH issue entirely)

```
docker build -t captureforge:0.1.0 -f env/Dockerfile .
# then run with the docker engine profile, e.g.:  captureforge run -profile test,docker
```

A laptop install is complete at this point: clone the repo, build the env (or image), and you can run the `test` profile and design real panels locally.

4.3 Shared HPC install (this cluster as the worked example)

For a multi-user cluster you install **once, centrally**, so anyone can `module load captureforge` and run from any node. The commands below use **this facility's cluster** as the concrete example; paths marked **[cluster-specific]** are what other sites change.

- **Cluster:** Ubuntu 24.04, 4 compute nodes. The install must be reachable **cluster-wide regardless of which node** a job lands on — so the environment, container, and references all live on shared storage.
- **Shared conda environment** — source the cluster conda, then create the env under the shared envs directory so all four nodes see it:

  ```
  source /hpc/opt/conda/etc/profile.d/conda.sh          # [cluster-specific] shared conda
  mamba env create -f env/captureforge.yml              # lands in
   ↪ /hpc/opt/conda/envs/captureforge
  conda activate captureforge                            # works from any node
  ```

  (Linux is native here — **no `CONDA_SUBDIR`/Rosetta**; that is a macOS-only step.)

- **Apptainer image** (native linux; Docker is usually disallowed on shared clusters):

  ```
  apptainer build /hpc/opt/conda/envs/captureforge.sif env/captureforge.def   #
   ↪ [cluster-specific] path
  ```

- **SLURM is global** here — **no special partition is required by default**. Keep the partition and account as **overridable parameters** (`--slurm_partition`, and an account via `--slurm_queue_opts`/ `clusterOptions` in `conf/hpc_slurm.config`) so users on partitioned sites can set them.
- **Access mechanism:** an **Apptainer image + Lmod modulefile + the `captureforge-run` wrapper**:

  ```
  module use /hpc/opt/captureforge/share/modulefiles   # [cluster-specific] install root
  module load captureforge/0.1.0
  captureforge-run -params-file my_panel.params.yaml   # add --slurm_partition/account only
   ↪ if needed
  ```

- `share/modulefiles/captureforge/0.1.0.lua` — Lmod modulefile stub (edit `install_root` / `refs_root`).
- `share/bin/captureforge-run` — wrapper that forwards to `nextflow run … -profile hpc_slurm,apptainer`.

The **full cluster install/deploy** — Apptainer build, centralized reference data + index, the modulefile, and the wrapper — is documented step-by-step in **`docs/hpc_deployment.md`**.

---

5 Preparing inputs

CaptureForge auto-detects the input format and normalizes **any one** of three on-ramps into a single internal target table (`stage1_targets/targets.tsv`). The logic that turns genes into labelled regions is **Stage 1 of the pipeline itself** — no prior project file or script is ever required. The schema is documented in `assets/gene_list_schema.md`.

5.1 On-ramp 1 — Plain-text symbols (zero friction)

One gene symbol per line. Blank lines and # comments are ignored. Applies the **default design classes** (`params.default_classes` = `coding,promoter`).

```
# my_genes.txt
TP53
EGFR
PTEN
```

5.2 On-ramp 2 — CSV/TSV spec (the reusable format)

Header **gene** is required; all others optional. Boolean-ish columns accept `1/0`, `yes/no`, `true/false`.

column	meaning
`gene` (**required**)	gene symbol (aliases in parentheses tolerated)
`promoter`	add the strand-aware promoter window
`regulatory`	intersect the Regulatory-Build / cCREs over the locus
`cnv`	add even-tiled CNV windows over the locus
`str`	tile unique flanks only (skip the repeat core)
`extra_regions`	extra intervals `chr:start-end` separated by `;`
`anchors`	rsIDs / variant tokens separated by `,` or `;`

```
gene,promoter,regulatory,cnv,str,extra_regions,anchors
TP53,1,0,0,0,,
EGFR,0,0,1,0,,
BRCA1,1,1,0,0,,rs1799966
PTEN,1,1,0,1,,
```

5.2.1 Scaffolding a spec from bare symbols

`bin/init_gene_list.py` turns a bare symbol list into an editable spec CSV pre-filled with the default classes, ready for you to add `promoter/regulatory/cnv/str/anchor` intent per gene:

```
captureforge panel my_genes.txt -o my_panel.spec.csv
# then edit my_panel.spec.csv and run with --input my_panel.spec.csv
```

5.3 On-ramp 3 — Rich curated workbook (`.xlsx`)

Reads the sheet named by `--excel_sheet` (default `Candidate Panel (expanded)`); the header row is detected automatically (the row containing a `Gene` cell). Recognized columns: `Gene` (required), `Variant class`, `Key variants to capture`, `Locus`, `Burden group`.

5.3.1 How `Variant class` text maps to design classes

The mapping is case-insensitive substring matching (`coding` is always added):

text contains	adds class
`reg`, `promoter`, `enhancer`, `regulatory`, `haplotype`	`promoter` (+ `regulatory` if `enhancer`/`distal`/`regulatory`)
`cnv`, `copy number`, `structural`, `deletion`, `duplication`	`cnv`
`str`, `repeat`, `tandem`, `microsatellite`, `expansion`	`str`
anything else (`coding`, `LoF`, `burden`, …)	`coding`

5.3.2 Aliases and anchors

Symbols written `A (B)` are resolved to whichever of `A`/`B` is a valid current symbol in the GTF (`ERBB2 (HER2)`→`ERBB2`, `TP53 (p53)`→`TP53`). Unresolvable symbols cause a **loud failure** with the offending list. `rs\d+` tokens in `Key variants to capture` / `anchors` are extracted as anchor targets (resolved to coordinates via a dbSNP VCF or a `--rsid_bed`; see *Reference data*).

5.4 Worked tutorial — starting from a gene name (p53 → TP53)

Suppose all you have is a gene you care about: **p53**. This walkthrough takes you from that name to an order package, and shows where each on-ramp fits.

5.4.1 Step 1 — get the symbol right (the most common trap)

`p53` is a **protein nickname**, not a gene symbol. CaptureForge resolves **HGNC/Ensembl gene symbols** against the GTF, so the symbol you feed it is **TP53**. If you pass `p53`, Stage 1 will not silently guess — it **fails loudly** and lists the unresolvable symbol. That loud failure is a *feature*: it stops you designing baits for the wrong (or no) locus.

Other nickname/alias traps that bite people:

You might type	Real symbol	Note
`p53`	`TP53`	protein name ≠ gene symbol
`HER2` / `Neu`	`ERBB2`	receptor nickname
`PD-1`	`PDCD1`	protein nickname

If you are unsure, look the gene up on genenames.org (HGNC) or Ensembl and use the **approved symbol**. Aliases written in parentheses (e.g. `TP53 (p53)`) are tolerated and resolved, but the bare approved symbol is cleanest.

5.4.2 Step 2 — choose an on-ramp

(a) Plain text — zero friction. A one-line file is a complete, valid input on its own:

```
echo TP53 > tp53.txt
```

With no other intent specified, TP53 gets the **default design classes** (`params.default_classes = coding,promoter`): the canonical-transcript **coding exons (incl. UTRs, ±15 bp splice flank)** plus a strand-aware **promoter** window (TSS−2000..+200). That is often all you want for a single gene.

(b) Scaffolded CSV spec — when you want to control intent. Generate an editable spec from the symbol list with `bin/init_gene_list.py` (it pre-fills the default classes; `coding` is always implied, and `promoter` is on by default):

```
captureforge panel tp53.txt -o tp53_panel.csv
```

That writes a CSV with the standard header and one row per gene, pre-filled:

```
gene,promoter,regulatory,cnv,str,extra_regions,anchors
TP53,1,0,0,0,,
```

Now **edit it** to add the intent you want — for TP53, say, turn on the regulatory class and add the common **Pro72Arg** polymorphism (`rs1042522`) as an anchor:

```
gene,promoter,regulatory,cnv,str,extra_regions,anchors
TP53,1,1,0,0,,rs1042522
```

Each column maps to a **Stage 1 design class**:

column	what it adds in Stage 1
(implicit)	`coding` — canonical exons ±15 bp splice flank (always)
`promoter=1`	strand-aware promoter window (TSS−2000..+200)
`regulatory=1`	intersect the Ensembl Regulatory Build / cCREs over the locus (needs `--regulatory_gff`)
`cnv=1`	even-tiled windows across the whole locus (read-depth CNV)
`str=1`	tile the unique flanks only (repeat core removed at masking)
`anchors=rs1042522`	the rsID ±150 bp, guaranteed ≥2 baits (needs `--rsid_bed` or a dbSNP VCF)

(c) Rich Excel workbook — only when a curated panel already exists in spreadsheet form (the `Variant class` → design-class mapping above). For a single new gene you do **not** need this; use (a) or (b).

5.4.3 Step 3 — make sure references exist

You need a no-alt genome FASTA, a matching GTF, and a specificity index built from that genome (see *Reference data*). For human GRCh38, fetch the optional tracks once:

```
bin/fetch_references.sh --outdir refs gtf
captureforge index \
    --fasta refs/GRCh38.primary_assembly.fa \
    --aligner blast \
    --out $PWD/refs/grch38_noalt
```

5.4.4 Step 4 — run

```
captureforge run -profile local \
    --input tp53_panel.csv \
    --genome_fasta refs/GRCh38.primary_assembly.fa \
    --gtf refs/Homo_sapiens.GRCh38.112.gtf.gz \
    --specificity_db $PWD/refs/grch38_noalt \
    --specificity_aligner blast \
```

```
--rsid_bed refs/anchors.rsid.bed \
--regulatory_gff refs/Homo_sapiens.GRCh38.regulatory_features.gff.gz \
--outdir results/tp53
```

The real parameter is **`--input`** (it auto-detects plain-text / CSV / Excel). `--regulatory_gff` and `--rsid_bed` are only needed because we turned on `regulatory` and added an anchor; drop them for the plain-text default run.

5.4.5 Step 5 — read the outputs

Under `results/tp53/` you get the same deliverables as any run (see *Outputs explained*): `order_package/baits.fasta` and `baits.csv` (the order-ready baits + metadata), `stage9_qc/final_covered_targets.bed` (the as-built handoff), and `qc_dashboard.html` (open it to see the per-gene track for TP53 with its exon lane, the GC/Tm windows, and the bait-construct schematic).

5.4.6 Honest caveat — one small gene is not a pool

TP53 is ~25 kb with ~11 coding exons; on its own it yields only **several hundred baits** — far below oligo-pool synthesis minimums (the smallest tier in `pool_tiers` is **12,000**). So "start from a gene name" is really how you **add a gene to a larger panel**: capture economics only work when **tens to hundreds of genes share one pool**. Build your CSV spec up to the full gene set, then run once — the pool cost is shared across every gene in the file.

---

6 Reference data

CaptureForge reference paths are parameters; you supply at minimum a **genome FASTA** and a **GTF**. Optional tracks (RepeatMasker, simpleRepeat, segmental dups, a regulatory build, dbSNP) improve masking and anchors but are not required.

6.1 Fetching human GRCh38 references

`bin/fetch_references.sh` downloads and **checksums** each reference and writes a provenance manifest. It is **GRCh38/human-specific** (Ensembl `homo_sapiens` + UCSC hg38). Heavy downloads never run automatically.

```
bin/fetch_references.sh --outdir refs gtf                # only the Ensembl r112 GTF (~50 MB)
bin/fetch_references.sh --outdir refs rmsk simplerepeat  # STR/repeat masks
bin/fetch_references.sh --outdir refs all                # everything (heavy!)
```

For **non-human** organisms, fetch the genome FASTA and GTF yourself (Ensembl / Ensembl Genomes / PlasmoDB) and pass them with `--genome_fasta` / `--gtf` (see *Running on different organisms*).

6.2 The no-alt reference invariant (read this)

> **Specificity must run against the NO-ALT primary assembly, and the specificity index must be the SAME reference as the design genome.**

If you index a *toplevel* / *with-alt* reference, genes in the MHC and other alt-haplotype regions will look non-specific against **their own alternate-haplotype copies** — for example, an MHC-region gene can appear 100% non-specific when its "off-targets" are its own copies on `HSCHR6_MHC_*` scaffolds. CaptureForge **enforces** the no-alt rule: Stage 4 (`specificity_filter.py`) fails loudly (exit 3) if the specificity reference contains alt/decoy/HLA contigs (`HSCHR*`, `*_MHC_*`, `HLA-*`, `*_alt`, `*_decoy`, `hs38d1`, EBV) and cross-checks that the db contigs are a subset of the design genome. On a clean primary assembly the check reports the contig count and passes (e.g. *"reference invariant OK: no alt/decoy/HLA"*).

6.3 Building the specificity index

Build the index **from the no-alt design genome**:

```
# BLAST (no index-build wait; convenient for local runs)
captureforge index \
    --fasta GRCh38.primary_assembly.fa \
    --aligner blast \
    --out $PWD/refs/grch38_noalt
# bowtie2 (native, recommended on the HPC)
captureforge index \
    --fasta GRCh38.primary_assembly.fa \
    --aligner bowtie2 \
    --out $PWD/refs/grch38_noalt_bt2
```

`--specificity_db` is referenced **in place** (not staged into work dirs), so it **must be an absolute path** to the index prefix.

---

7 Configuration

7.1 Profiles

Profile	Executor / use	Compose with
`test`	tiny mini-genome fixture; runs on a laptop in ~2 min	`,conda` / `,docker`
`local`	full pipeline on one machine (any Linux/macOS)	`,conda` / `,docker`
`hpc_slurm`	SLURM executor; big slots for the heavy specificity step	`,apptainer`

Container-engine profiles `conda`, `docker`, `apptainer` are composed, e.g. `-profile hpc_slurm,apptainer`.

7.2 Key parameters (real defaults from `nextflow.config`)

Param	Default	Meaning
`bait_len`	`120`	bait length (nt)
`canonical_source`	`mane`	transcript pick: `mane` → `ensembl_canonical` → `longest` fallback
`splice_flank`	`15`	bp added around each exon
`promoter_upstream` / `promoter_downstream`	`2000` / `200`	TSS−2000 .. TSS+200
`anchor_flank`	`150`	bp each side of an rsID anchor
`default_classes`	`[coding, promoter]`	applied to a bare symbol list
`tiling`	per-class `density` / `step_frac` / `even`	tiling policy (coding ½-step, promoter/reg ⅓, cnv even, str ⅓ flanks)
`specificity_aligner`	`bowtie2`	`bowtie2` | `bwa` | `blast`
`specificity_db`	`null`	aligner index / BLAST db prefix (**absolute path**, no-alt)
`offtarget_min_identity`	`0.90`	secondary-hit identity threshold
`offtarget_min_lenfrac`	`0.50`	secondary-hit length fraction

Param	Default	Meaning
`offtarget_max_hits`	1	tolerated off-target hits before reject
`gc_min` / `gc_max`	`0.25` / `0.70`	GC window (fraction)
`gc_max_promoter`	`0.80`	relaxed upper GC for promoter/regulatory
`homopolymer_max`	10	max homopolymer run
`tm_min` / `tm_max`	`60.0` / `90.0`	nearest-neighbour Tm window (°C)
`retile_max_passes`	3	max gap re-tiling passes
`coverage_target`	`0.95`	target fraction of **masked** target to cover
`bait_format`	`ivt_rna`	`dna_pool` | `ivt_rna` | `ready_to_use`
`synth_max_len`	170	vendor synthesis length ceiling (nt)
`per_oligo_price`	`0.00200`	USD per oligo (**placeholder**)
`pool_tiers`	`[12000, 92000, 244000]`	oligo-pool size tiers
`outdir`	`results`	output directory

Some advanced parameters are present for documentation but are **not currently wired** to a stage — see *Troubleshooting → Known doc-vs-behaviour notes*.

---

8 Running the pipeline

Recommended: `captureforge run …` (and `captureforge panel` / `captureforge index`) after `pip install -e .`. The developer raw form still works from a clone — the repo bundles a launcher, so `./.tools/nextflow run main.nf -profile local,conda …` runs the pipeline directly (or system `nextflow` if installed). `captureforge run` prefers the bundled launcher and removes the boilerplate.

8.1 Quickstart — the `test` profile (10 genes, ~2 min)

The fixture (a generic TP53/EGFR/BRCA1/… mini-genome) ships pre-built in `test/data/`, so this runs straight from a clone — no build step, no local GRCh38 required:

```
conda activate captureforge   # arm64: env must be active (captureforge run warns otherwise)
captureforge run -profile test  # or: -profile test,conda
bash test/check_outputs.sh                    # sanity-check; expects all 6 design classes
```

To rebuild/refresh the fixture (reproducibility, or after editing `test/data/test_genes.csv`), run `test/data/make_test_reference.sh` against your own local GRCh38 — it is **not** required for the quickstart above.

8.2 Full run — `local`

```
captureforge index \
    --fasta GRCh38.primary_assembly.fa \
    --aligner blast \
    --out $PWD/refs/grch38_noalt
conda activate captureforge   # captureforge run warns if the env is not active
captureforge run -profile local -params-file params.full.yaml
```

8.3 Full run — `hpc_slurm` with Apptainer

Build the container once (Apptainer and Singularity are interchangeable — `singularity build` takes the same arguments), then run through the same `captureforge run` front door used locally, pointing `--pipeline` at the installed `main.nf`:

```
apptainer build captureforge.sif env/captureforge.def
captureforge run \
    --pipeline /shared/apps/captureforge/main.nf \
    -profile hpc_slurm,apptainer \
    -params-file my_panel.params.yaml \
    --genome_fasta $CF_REFS/GRCh38.primary_assembly.fa \
    --gtf $CF_REFS/GRCh38.112.gtf.gz \
    --specificity_db $CF_REFS/grch38_noalt_bt2 \
    --specificity_aligner bowtie2 \
    --rmsk_bed $CF_REFS/rmsk.GRCh38.txt.gz \
    --simplerepeat_bed $CF_REFS/simpleRepeat.GRCh38.txt.gz \
    --rsid_bed $CF_REFS/anchors.rsid.bed \
    --slurm_partition <queue> \
    --scratch_dir $TMPDIR \
    --outdir results
```

> On a **deployed** cluster, `module load captureforge` puts the launcher on PATH and resolves the install path, the container image, and the `hpc_slurm,apptainer` profile from the modulefile — so the invocation reduces to `captureforge-run -params-file my_panel.params.yaml --slurm_partition <queue>` (the Lmod wrapper), with no `--pipeline` needed. The exact PATH/module mechanism, the shared install path, and the partition/account are **finalized at HPC deployment** — see *Shared HPC install* above and `docs/hpc_deployment.md`. Keep `--slurm_partition <queue>` as a placeholder for your site's queue.

---

9 Running on different organisms

CaptureForge is organism-agnostic: you change **inputs and parameters**, not code. The same Stage 1–10 pipeline runs for any species given a genome FASTA + GTF and a specificity index built from that genome. Below are real, runnable command shapes for three organisms plus a general checklist.

The two things that most often need per-organism attention are: (1) the **canonical-transcript source** (MANE exists only for human) and (2) the **GC/Tm windows** (an AT-rich genome will reject most baits at the human default GC window).

9.1 Human (GRCh38) — the baseline

The human baseline uses MANE Select canonical transcripts and the full set of human tracks.

```
captureforge run -profile local \
    --input my_panel.spec.csv \
    --genome_fasta refs/GRCh38.primary_assembly.fa \
    --gtf refs/Homo_sapiens.GRCh38.112.gtf.gz \
    --specificity_db $PWD/refs/grch38_noalt \
    --specificity_aligner blast \
    --rmsk_bed refs/rmsk.GRCh38.txt.gz \
    --simplerepeat_bed refs/simpleRepeat.GRCh38.txt.gz \
    --rsid_bed refs/anchors.rsid.bed \
    --canonical_source mane
```

9.2 Mouse (GRCm39, Ensembl) — MANE-absent fallback

Mouse has **no MANE**. With `--canonical_source mane`, CaptureForge falls back automatically (MANE_Select → Ensembl_canonical → longest); the mouse Ensembl GTF carries `Ensembl_canonical` tags, so you can also set it explicitly. Mouse has an Ensembl **Regulatory Build** (pass it as a GFF) but you will supply your own variant BED for anchors.

```
# fetch GRCm39 primary assembly + GTF from Ensembl yourself, then:
captureforge index \
    --fasta GRCm39.primary_assembly.fa \
    --aligner bowtie2 \
    --out $PWD/refs/grcm39_bt2

captureforge run -profile local \
    --input mouse_panel.spec.csv \
    --genome_fasta refs/GRCm39.primary_assembly.fa \
    --gtf refs/Mus_musculus.GRCm39.112.gtf.gz \
    --specificity_db $PWD/refs/grcm39_bt2 \
    --specificity_aligner bowtie2 \
    --canonical_source ensembl_canonical \
    --regulatory_gff refs/Mus_musculus.GRCm39.regulatory_features.gff.gz \
    --rsid_bed refs/mouse_anchors.bed   # user-supplied (chrom start end id), optional
```

Caveats: `fetch_references.sh` is human-only — fetch mouse refs manually. UCSC `rmsk`/`simpleRepeat` use `mm39` (not the hg38 paths in the script); if you don't supply them, masking falls back to **dustmasker** (low-complexity) and STR-core removal is weaker (still functional). dbSNP anchors require a user BED.

9.3 Plasmodium falciparum — non-model, AT-rich genome

P. falciparum is ~**81% AT** (≈19% GC). Two things change materially:

- **There is no MANE and usually no `Ensembl_canonical` tag** in PlasmoDB/Ensembl-Protists GTFs → canonical selection falls back to the **longest** transcript. Set `--canonical_source longest` explicitly.
- **The default GC window (0.25–0.70) rejects most baits**, because most *P. falciparum* sequence has GC below 25%. You must **widen and shift the GC window down**, and lower the Tm floor (AT-rich baits melt lower). Without this, Stage 5 will reject the large majority of candidates.

```
# Gene models from PlasmoDB / Ensembl Protists; primary assembly (no alts):
captureforge index \
    --fasta PlasmoDB-Pf3D7_Genome.fasta \
    --aligner bowtie2 \
    --out $PWD/refs/pf3d7_bt2

captureforge run -profile local \
    --input pf_drug_resistance.spec.csv \
    --genome_fasta refs/PlasmoDB-Pf3D7_Genome.fasta \
    --gtf refs/PlasmoDB-Pf3D7.gtf \
    --specificity_db $PWD/refs/pf3d7_bt2 \
    --specificity_aligner bowtie2 \
    --canonical_source longest \
    --gc_min 0.15 --gc_max 0.55 \
    --gc_max_promoter 0.60 \
    --tm_min 50 --tm_max 85
```

A *P. falciparum* drug-resistance panel (e.g. *pfcrt*, *pfmdr1*, *dhfr*, *dhps*, *kelch13*) is designed exactly this way — coding classes for the resistance genes, the widened GC/Tm windows above, and a specificity index built from the Pf3D7 primary assembly.

Caveats: no regulatory build and no dbSNP — `regulatory` flags produce nothing (but **promoter windows still work**, since they come from the GTF TSS), and anchors require a user-supplied BED. No UCSC repeat

tracks — masking is **dustmasker-only** (which is actually well-suited to the AT-rich, low-complexity-rich Pf genome). The no-alt invariant still applies: Pf has no alt haplotypes, so the check passes.

9.4 "Any organism" checklist

Before a new-organism run, confirm:

1. **Minimum inputs:** a gene list; a **no-alt primary-assembly** genome FASTA; a matching GTF with `gene_name` attributes; a specificity index **built from that FASTA** (absolute path).
2. **Canonical source:** `mane` only for human; use `ensembl_canonical` (most Ensembl species) or `longest` (non-model) otherwise.
3. **GC/Tm windows:** check the genome's GC content. AT-rich (Plasmodium, many bacteria) → lower `gc_min`, lower `gc_max`, lower `tm_min`. GC-rich → raise the window.
4. **Optional tracks:** supply `rmsk_bed`/`simplerepeat_bed` (correct assembly!) and `regulatory_gff` if they exist; otherwise masking is dustmasker-only and `regulatory` classes are empty (promoter still works).
5. **Anchors:** supply `--rsid_bed` (or a tabix dbSNP VCF via `--dbsnp_vcf`) only if you flagged anchors.
6. **Contig naming:** the genome FASTA, GTF, and any tracks must use the **same** contig names; UCSC `chr`-prefixed tracks are normalized to Ensembl naming automatically, but mismatched assemblies are not.

---

10 Outputs explained

All outputs land under `results/<outdir>/`. The deliverables a user cares about:

10.1 `order_package/`

- **`baits.fasta`** — the final, order-ready bait constructs (for `ivt_rna`, each is `handle5 + T7 + bait + handle3`). FASTA header carries `copies=` and `fmt=`.
- **`baits.csv`** — one row per unique bait, full metadata. Columns:

column	meaning
`bait_id`	stable unique id (`CF_NNNNNN|gene|class`)
`sequence`	the final synthesis construct (handles + bait for `ivt_rna`)
`copy_number`	how many times this bait is represented in the pool (boosting; see Stage 7)
`gene`	source gene(s)
`class`	design class(es) the bait came from
`source_region`	the genomic region(s) it tiles
`strand`	+ / −
`GC`	GC fraction of the **bait core** (the hybridizing 120-mer)
`Tm`	nearest-neighbour Tm (°C) of the core
`off_target_flag`	1 if the bait retains a tolerated paralog/off-target hit, else `0`

- **`order_generic.tsv`** — a vendor-neutral order table (`Pool_Name, Probe_ID, Sequence, Replicates`).
- **`MANIFEST`** — md5 checksums of the key outputs.

- **`provenance.json`** — tool versions, reference releases, `git_commit`, `design_genome`, and `specificity_reference` (so every output records exactly which reference produced it).

10.2 `stage9_qc/`

- **`final_covered_targets.bed` — the handoff to a downstream analysis pipeline.** This is the **AS-BUILT** coverage: the intersection of your raw targets with what the final baits actually cover, labelled `gene|class`. **It is not the raw target set** — measure downstream variant calls against this file, because it represents the regions you can actually expect reads in.
- `metrics.json`, `coverage_by_gene.tsv`, `coverage_by_class.tsv` — machine-readable QC.

10.3 `qc_dashboard.html` and `figures/`

A self-contained HTML dashboard (open in a browser) embeds every figure and the key metrics tables. All figures are also written as **PNG + SVG** under `figures/`. Per-stage intermediates live under `stage1_targets/` … `stage8_synthesis/`; `pipeline_info/` holds the Nextflow `report/timeline/trace/dag`.

---

11 Interpreting QC and coverage

This is the most important section for making good decisions.

11.1 Mappable vs raw coverage

CaptureForge reports coverage two ways:

- **mappable coverage** = covered ÷ **masked (tileable)** target — design efficiency over sequence you can actually bait.
- **raw coverage** = covered ÷ **raw** target — the honest end-to-end number over the *nominal* footprint.

A typical panel shows a **high mappable** number (often above 90%) and a **lower raw** number. In a repeat-heavy or AT-rich panel the gap can be dramatic — for illustration, mappable ≈ 90% against raw ≈ 67% (the bundled `test` panel's gap is much smaller). **This is by design.** The gap is sequence intentionally removed by masking — repeats, segmental duplications, STR cores, Alu elements — which is **genomically uncapturable** by hybridization. A high mappable number with a lower raw number is the *expected, honest* result, not a failure.

To see *how* each gene's baits are composed by design class, the per-gene-by-class view stacks bait counts per gene (counts from `baits.csv`, multi-class baits attributed to their primary class so the bars sum to the unique-bait count). It makes the design intent obvious at a glance: a gene flagged `str` is dominated by its unique-flank baits, a `cnv` gene by whole-locus tiling, and a plain coding gene by `coding+promoter`. The full gene × class matrix is written to `figures/baits_per_gene_by_class.tsv`.

11.2 The three-tier per-gene honesty

Not all "incomplete" coverage means the same thing. Genes fall into three tiers:

1. **Cleanly covered** — unique sequence, high mappable coverage (most of a panel).
2. **Paralog-limited coding** — only the *uniquely-baitable fraction* is covered; the rest is too similar to a paralog to bait specifically (a gene in a multi-copy family, or with a near-identical paralog elsewhere in the genome). Baits that retain one tolerated paralog hit are tagged `off_target_flag=1` in `baits.csv`.
3. **CNV breakpoint-blind** — for CNV genes, depth bins are evenly spaced *except* across segmental-dup / LCR regions, which are exactly where the structural variation often occurs, leaving multi-kb gaps across those blocks.

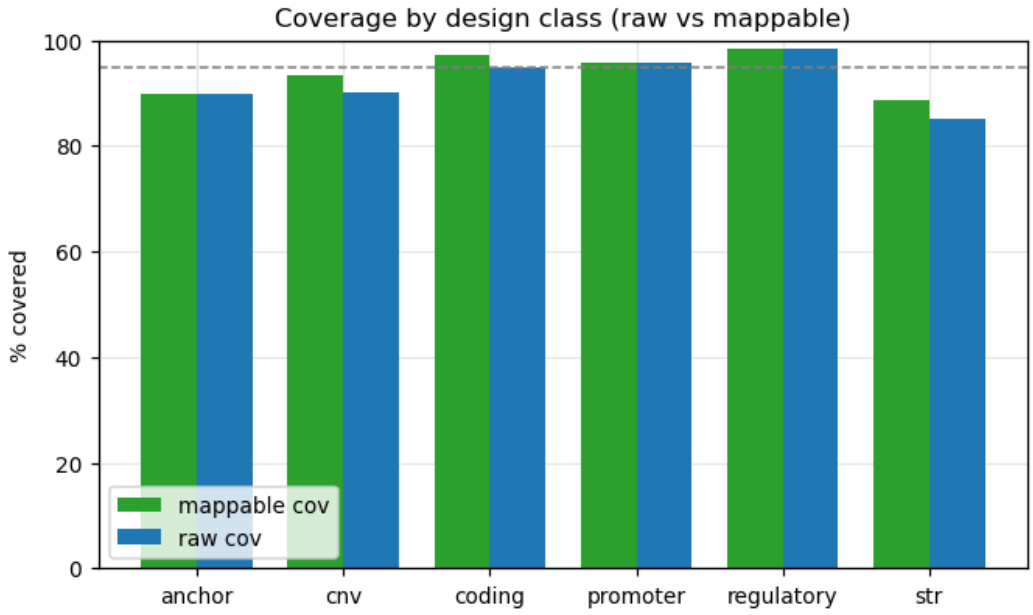


Figure 2: Coverage by design class. Green = mappable coverage (over tileable sequence), blue = raw coverage (over the nominal target). Read the green bars for design efficiency; the green–blue gap is masked/uncapturable sequence. `coding`/`promoter`/`str` classes are typically high mappable; `cnv` is usually lower because repeat-rich loci are partly masked and partly non-specific.

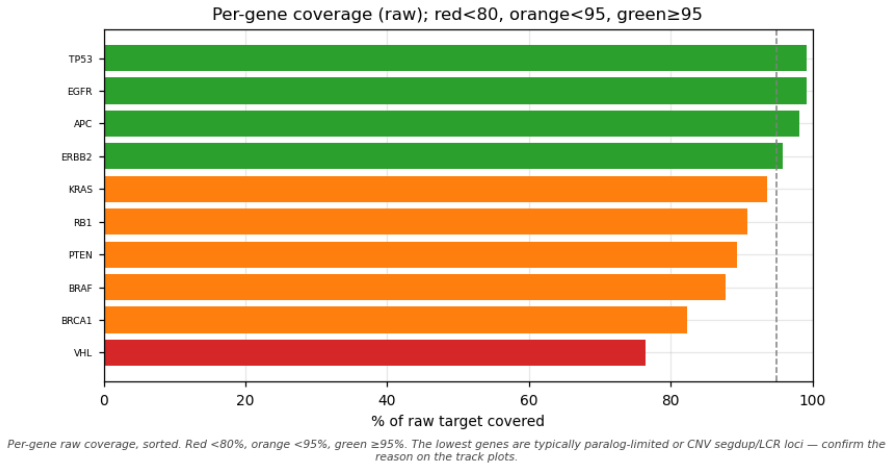


Figure 3: Per-gene raw coverage, sorted. Green ≥95%, orange <95%, red <80%. The red/orange genes are not “broken” — they are typically the paralog-limited and CNV/segdup loci discussed below. Use this to spot which genes need a second look, then confirm the reason on the track plots.

A gene that is **both** paralog-limited **and** breakpoint-blind is the kind most likely to need an **orthogonal assay** (MLPA / long-read) if it is central to your study.

11.3 CNV read-depth callability is the decision metric — not raw CNV coverage

For a CNV gene, the question is not "what % of the locus is baited" but "are the baited bins **evenly spaced enough** to call copy number from read depth?" CaptureForge reports per-CNV-gene bin spacing (`metrics.json` → `cnv_bin_spacing`): number of bins, median and **max inter-bin gap**, and a `depth_callable` flag. A CNV gene in clean, unique sequence is depth-callable (small max gaps); one whose locus is broken up by LCRs is not (multi-kb blind spots). A gene can have hundreds of bins and still be breakpoint-blind.

11.4 Reading a per-gene track plot — real vs uncapturable gaps

Each per-gene panel has three lanes: **target** (raw, colored by class), **tileable** (after masking), and **baits** (final). Compare the lanes:

- A gap in **baits** where **tileable** is also empty → the sequence was **masked away** (repeat/segdup) → **uncapturable, expected**, not a coverage failure.
- A gap in **baits** where **tileable** is present → a **real** gap (a bait was rejected for specificity or composition) → worth investigating.

Each panel now has **four lanes** — `target` (raw, class-colored), `exons` (the coding exons ±15 bp splice flank, in green, right under the target so you can check no exon is missed), `tileable` (post-mask), and `baits` (final) — and the gene label includes the **exon count** alongside the chromosome location.

11.5 Composition QC — GC and Tm

Stage 5 rejects baits outside the GC window (default 25–70%, 80% upper for promoters), Tm window (60–90 °C), homopolymer (>10), or low entropy. The histograms show the passing population sitting inside those windows; the dashed lines are the thresholds. **For an AT-rich organism these distributions shift left** and you must move the windows (see *Running on different organisms*), or almost everything is rejected.

> **What these histograms count.** They count **spec-pass candidate baits — one row per tiled position** (from `composition.tsv`); boosted **copies are NOT multiplied in**, so a boosted bait is counted once, not 2–4×. The tall spikes are therefore *expected*: dense tiling of a compositionally homogeneous region produces many near-identical baits at the same GC/Tm. What matters is that the **bulk of the distribution sits inside the GC 25–70% / Tm 60–90 °C windows**, not the spike heights.

11.6 Specificity QC — the off-target distribution

Stage 4 counts, per bait, how many off-target loci it hits above the identity/length thresholds. A unique bait has 0; one tolerated hit (≤ `offtarget_max_hits`) still passes; more are rejected. The long tail (baits hitting hundreds or thousands of loci) are repeat-derived candidates, correctly discarded.

11.7 Boosting / dedup — copy number

Stage 7 collapses identical bait sequences (summing copies) and adds extra **copies** over boost-flagged classes (promoter/regulatory, anchors, high-GC promoters). Copies increase a probe's *molar representation* in the pool; they do **not** add distinct oligos, so they do not change the pool size or tier.

11.8 Re-tiling convergence — when non-convergence is correct

Stage 6 iteratively tiles denser baits over gaps and re-checks them, up to `retile_max_passes`, aiming for `coverage_target` (0.95) of the **masked** target. A typical run climbs a few points across passes and then

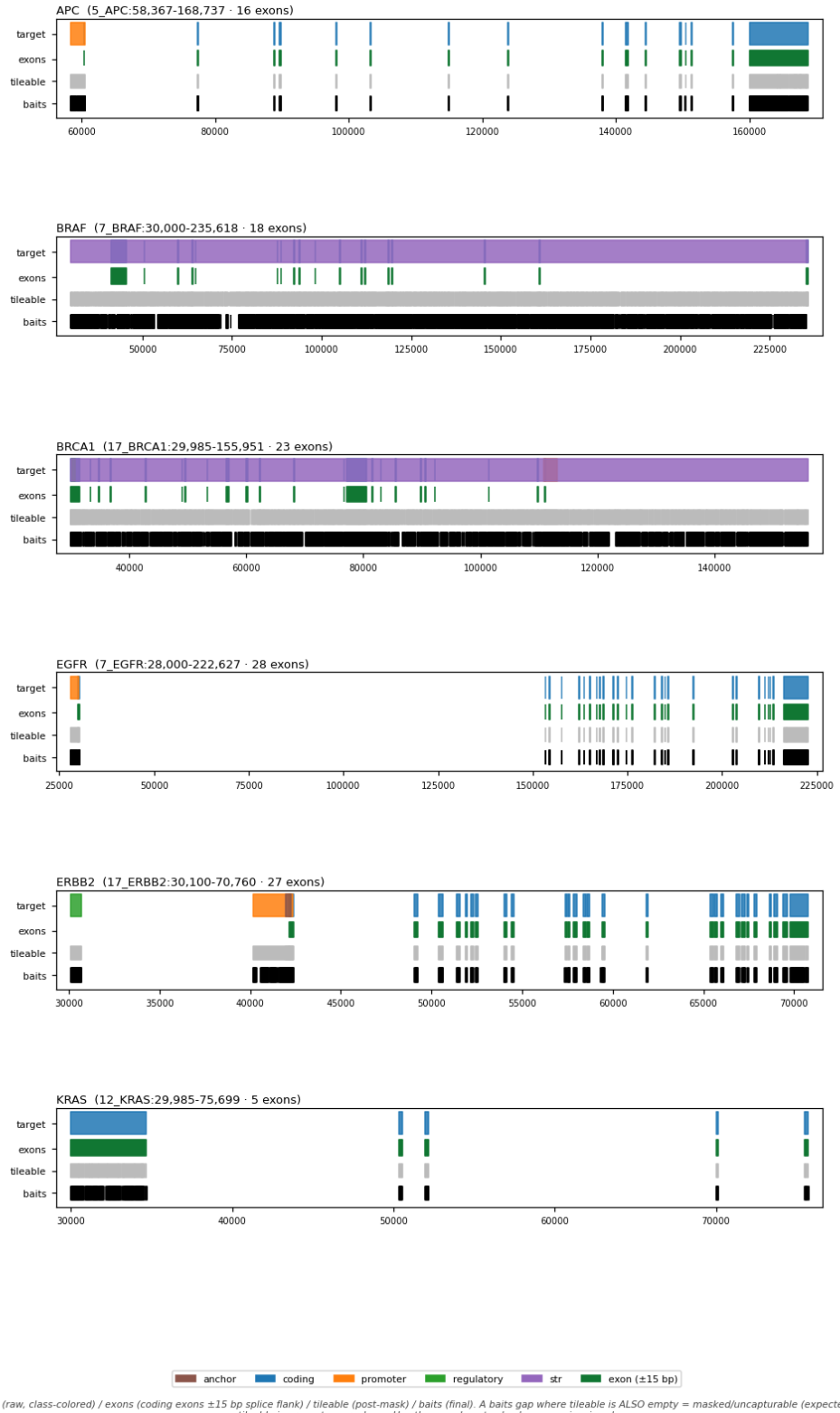


Figure 4: A clean page: the `baits` lane closely tracks the `target`, `exons`, and `tileable` lanes — every exon is well covered with only small, expected gaps. (Genes are laid out a few per page, spaced so coordinate ticks never collide with the next gene's title; the label shows the exon count.)



Figure 5: A second page of per-gene tracks. Where a gene carries a `cnv` or `str` class, the `target` lane spans the whole locus while the green `exons` lane shows where the coding exons sit; on a real run the `baits` lane thins out across any segmental-dup / LCR blocks that are unmappable. The exon lane lets you confirm which exons remain covered despite breakpoint-blind regions.

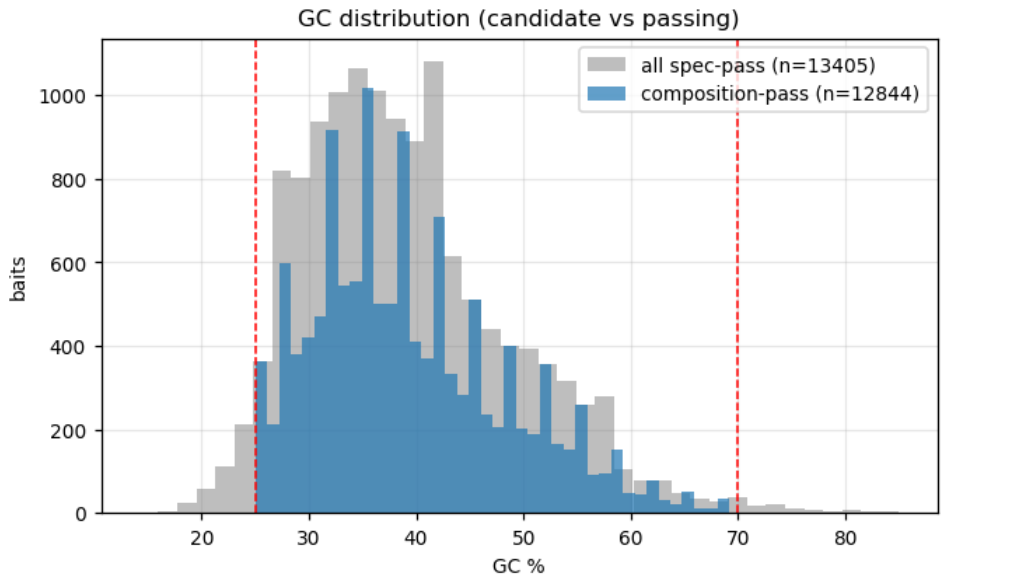


Figure 6: GC distribution (spec-pass candidate baits — positions, not copies). Grey = all specificity-passing candidates; blue = those that also pass composition. Dashed lines mark the 25–70% window. Spikes are expected; if the population sits *outside* the window (as for an AT-rich genome), widen/shift `gc_min`/`gc_max`.

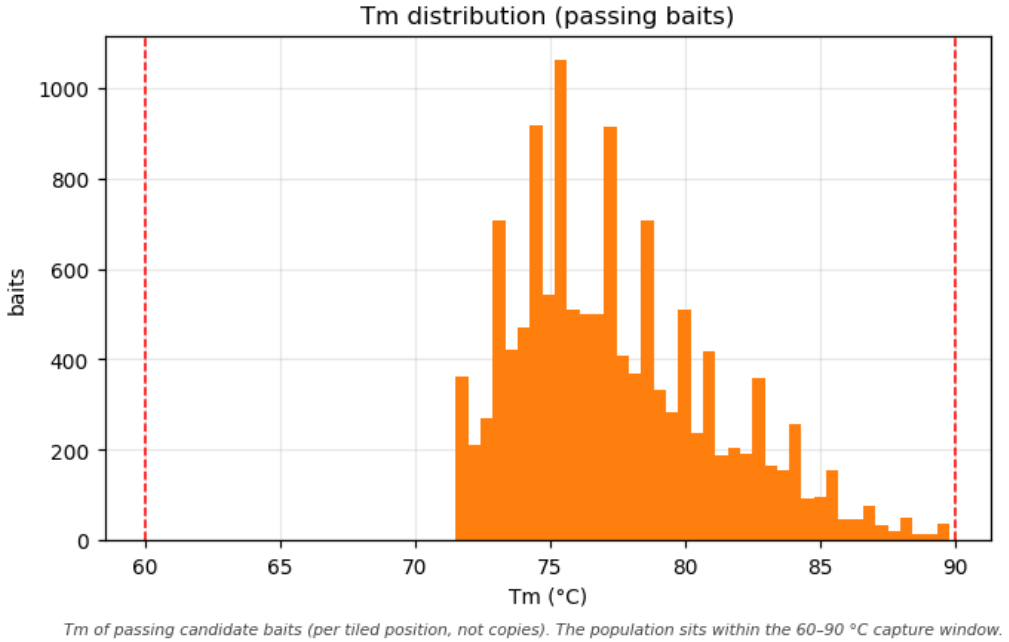


Figure 7: Tm distribution of passing candidate baits (positions, not copies), 60–90 °C window marked. AT-rich baits melt lower; if the population piles up below `tm_min`, lower it.

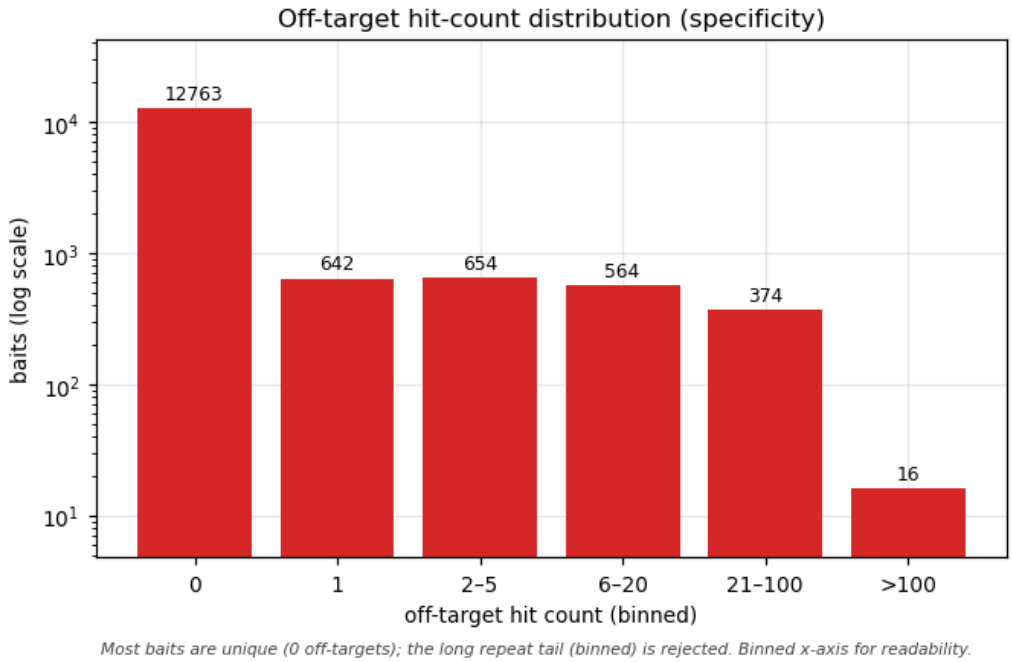


Figure 8: Off-target hit-count distribution, **binned x-axis** (0 / 1 / 2–5 / 6–20 / 21–100 / >100) with a log y-axis and counts above each bar. The bin at 0 are unique baits; the bin at 1 (tolerated, tagged `off_target_flag=1`) and 2–5 are paralog hits; the right-hand bins are repeat baits that are rejected. Binning keeps the 0-to-thousands range readable.

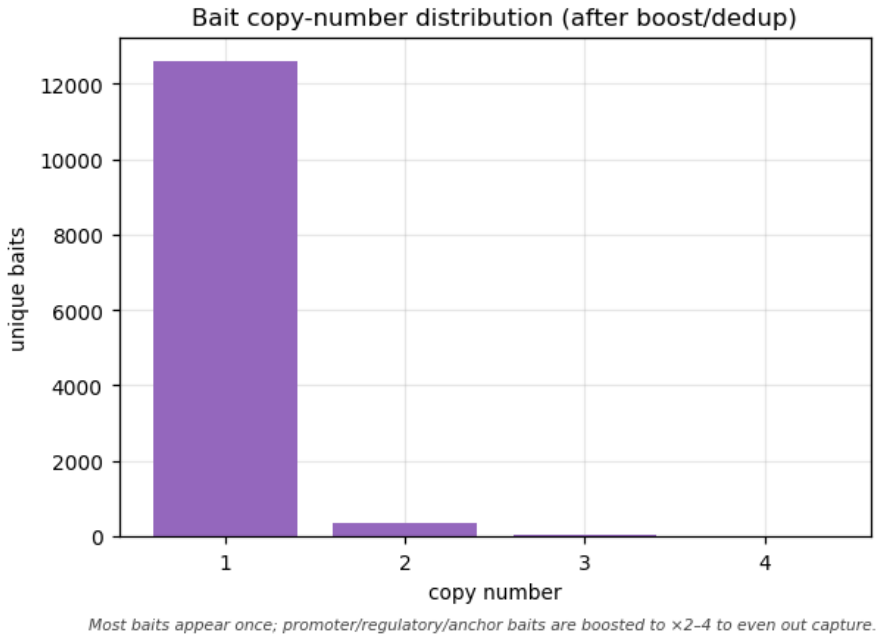


Figure 9: Bait copy-number distribution after boost/dedup. Most baits appear once (×1); promoter/regulatory and anchor baits are boosted to ×2–4 to even out capture efficiency.

stops without reaching 0.95 — and that is the **right** outcome: the remaining gaps are non-specific / repeat sequence that denser baiting cannot fix. **Non-convergence here is honest, not a bug** — forcing baits into those regions only adds off-target probes.

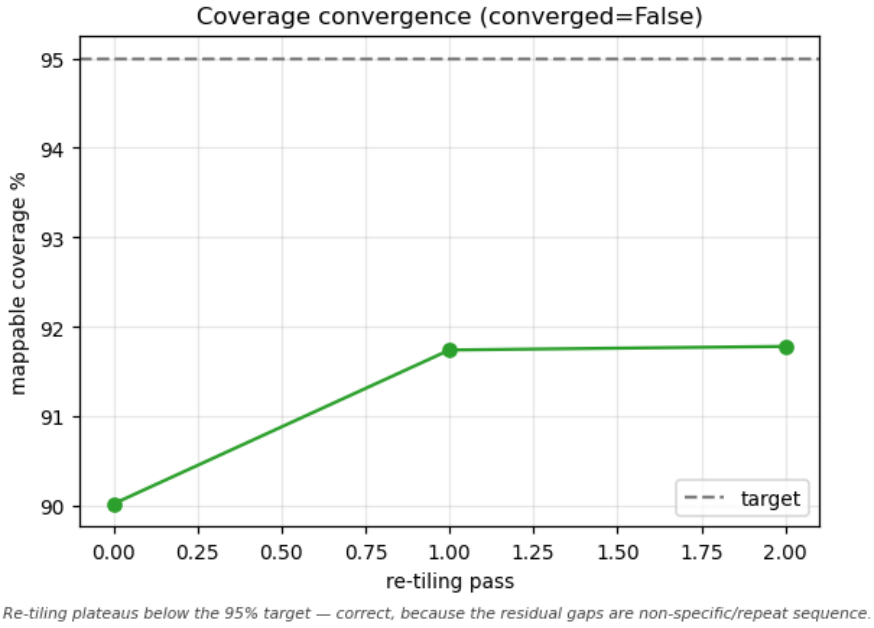


Figure 10: Coverage convergence across re-tiling passes. The curve plateaus below the target line because the residual gaps are genomically non-specific — the correct place to stop.

---

12 Cost and ordering

12.1 Pool cost and tiers

CaptureForge predicts an **oligo-pool synthesis cost** = unique baits × `per_oligo_price`, and the smallest oligo-pool **tier** the panel fits. As a worked example with the placeholder price, the bundled `test` panel of **12,963 unique baits** at **$0.00200/oligo** costs **$25.93**. That count **exceeds the smallest 12,000 tier**, so it does *not* fit there — it crosses the 12,000 line and lands in the **92,000** tier, and a *tripwire* fires. The tripwire is the point: it prevents the pipeline from silently spilling into a larger, more expensive tier, so you decide whether to trim the panel or accept the bigger pool.

> **What the pool cost is and is NOT.** It is the **synthesis cost of the oligo pool only**. It does **not** include per-sample assay/capture reagents, biotinylation/IVT, sequencing, or vendor setup fees, and the `$0.00200`/oligo and the tier list are **placeholders** — replace them with real vendor quotes.

---

13 Troubleshooting

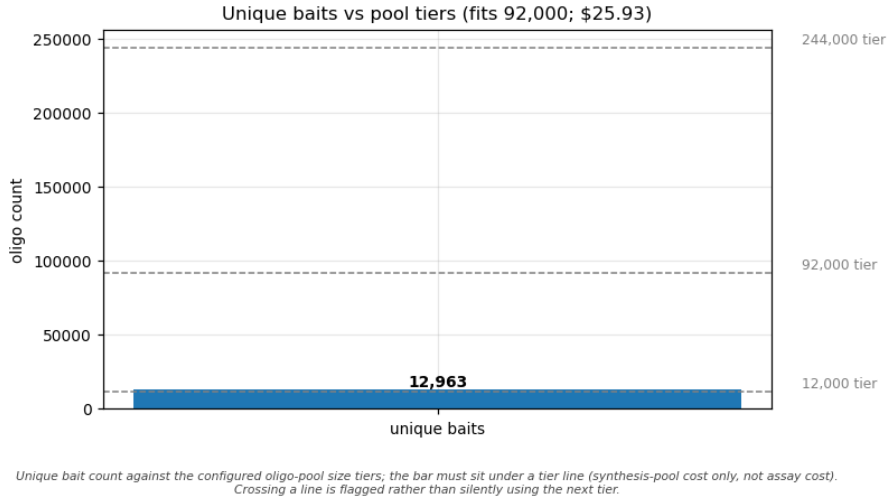


Figure 11: Unique bait count against the configured pool-size tiers. Here the bait-count bar crosses the smallest (12k) tier line and lands in the 92k tier, so the pipeline **flags** it (the tripwire) rather than silently using the next tier up. A count that stays under the smallest line would fit it with headroom.

Symptom	Cause	Fix
`ERROR: specificity reference contains ALT/decoy/HLA contigs` (exit 3)	You pointed `--specificity_db` at a toplevel/with-alt index	Rebuild from the **no-alt primary assembly** with `bin/build_specificity_index.sh`; see the reference-invariant section
`ERROR: specificity_db not found … must be an ABSOLUTE path`	Relative `--specificity_db`; it is referenced in place, not staged	Use an absolute path to the index prefix
A gene at **0% coverage**	Either a real paralog problem **or** a reference artifact	Check the off-target subjects: if they are alt-haplotype contigs (`HSCHR…`), it's a reference artifact (the no-alt invariant) — fix the index; if they are a distinct paralog gene, it's genuine paralog-limited coverage
Most baits rejected on a new organism	AT-rich (or GC-rich) genome vs the human GC/Tm window	Widen/shift `gc_min`/`gc_max`/`tm_min` (see Plasmodium example)
`python3: command not found` / `ModuleNotFoundError` on macOS	`/usr/bin/env python3` resolved to Homebrew python, not the env	`conda activate captureforge && export PATH="$CONDA_PREFIX/bin:$PATH"`, or run `-profile <x>,conda/,docker`
Specificity numbers differ local vs HPC	blastn (local) vs bowtie2 (HPC) detect off-targets slightly differently	Re-validate on the production engine before ordering; both are correct, the margins differ
Re-tiling "did not converge" warning	Residual gaps are non-specific/repeat	Expected — see the convergence section; not an error
PDF build fails (no LaTeX)	No `xelatex` on the machine	Install TeX Live / TinyTeX, or fall back to the HTML→PDF route (see `bin/build_manual.sh`)

13.1 Known doc-vs-behaviour notes

A few parameters exist in `nextflow.config` for documentation but are **not currently wired** to a stage; the script defaults apply instead. Setting them has no effect:

- `genmap_kmer`, `genmap_errors`, `genmap_min_mappability` — GenMap is **not** invoked; specificity uses alignment (blast/bowtie2) only.
- `mask_low_complexity`, `relax_lowcplx_promoter`, `str_skip_repeat_core` — `modules/masking.nf` does not pass these; `apply_masking.py` defaults them on (which matches the config defaults).
- `dust_threshold` — the low-complexity cutoff actually used is `composition_filter.py --min-entropy` (default 1.2), not `dust_threshold`.
- `tiling[*].density` — informational; the tiler uses `step_frac` and `even` (boosting handles copies).

---

14 From design to order

14.1 Choosing a bait format (`bait_format`)

Format	What you order	Implication
`ivt_rna` (default)	DNA oligos = `handle5 + T7 + bait + handle3` (170 nt)	Amplify by PCR, transcribe to **biotinylated RNA** baits in-house (T7). Lowest cost per capture; some wet-lab work
`dna_pool`	the bait cores only	Synthesized DNA oligo pool; simplest, but DNA baits and no amplification
`ready_to_use`	bait sequences + target BED	For a finished-bait vendor service (Arbor myBaits etc.)

The schematic below shows what the synthesized oligo looks like for each format (segments scaled by length in nt). For `ivt_rna` the 120 nt bait (probe) is flanked by a 5′ universal PCR handle, a T7 promoter, and a 3′ PCR handle (170 nt total): PCR-amplify the pool, then in-vitro-transcribe with T7 into biotinylated RNA baits. `dna_pool` and `ready_to_use` are the bait core only. The handle/T7 sequences are configurable (`pcr_handle_5`, `t7_promoter`, `pcr_handle_3`); the lengths shown come from the run's stage-8 summary.

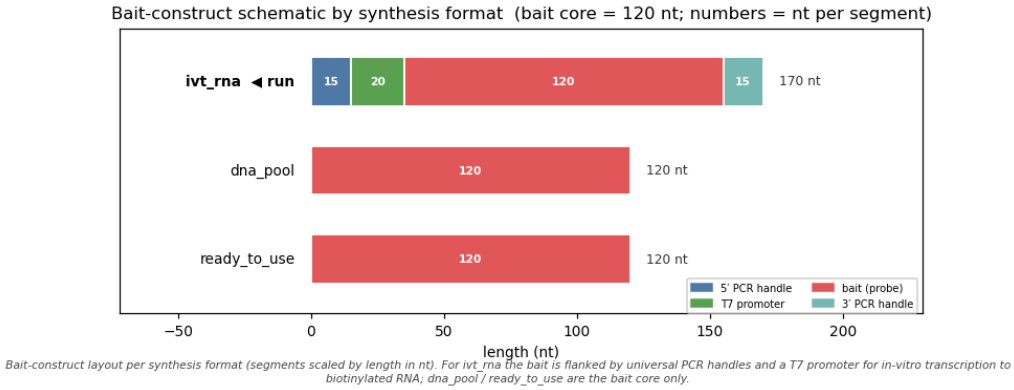


Figure 12: Bait-construct schematic for every synthesis format CaptureForge can produce; segments are scaled by length in nt and the format used for this run is marked ◀ `run`. ivt_rna = 5′ PCR handle (15) + T7 promoter (20) + bait (120) + 3′ PCR handle (15) = 170 nt; dna_pool / ready_to_use = the 120 nt bait core.

14.2 Mapping to a synthesizer template

CaptureForge emits a **generic** order package. Map `baits.csv` / `order_generic.tsv` to your vendor's template before ordering (CaptureForge never contacts a vendor):

Vendor	Product	Note
Twist	oligo pool / custom panel	accepts FASTA + target BED
IDT	xGen Custom / Lockdown	120 nt biotinylated probes
Agilent	SureSelect / custom baits	120 nt RNA baits
Arbor Biosciences	myBaits	80–120 nt, IVT RNA (`ready_to_use`)
GenScript / Twist	oligo pools	for `ivt_rna` (T7 + handles)

The `baits.csv` columns cover what every template needs; reshape/rename per the vendor. See `assets/vendor_templates/`.

14.3 Pre-order checklist

1. **Validate the production specificity engine** — if you designed with blastn locally but will deploy bowtie2 on the HPC, re-run specificity on the production engine and diff.
2. **Decide orthogonal assays** for breakpoint-blind / paralog-limited loci (the genes flagged in the QC).
3. **Replace placeholder costs** (`per_oligo_price`, `pool_tiers`) with real vendor quotes.
4. **Confirm the Tm window** matches your actual capture protocol (the default assumes capture, not PCR).
5. **Confirm accepted losses** — e.g. an exon that falls entirely inside a high-copy SINE/Alu element is intentionally **not baited** (baiting it would pull down ~1M genomic Alu copies).

---

15 Extending CaptureForge

- **Adjusting design classes / params** — most behaviour is data-driven via `nextflow.config` params and the per-class `tiling` map; change tiling density, flanks, thresholds, or GC/Tm windows without touching code.
- **Roadmap (NOT yet implemented):**
  - **Curated CNV-breakpoint targeting** — replace whole-locus CNV tiling with targeted breakpoint baits (needs curated breakpoint coordinates).
  - **Paralog-aware design** — design baits at paralog-discriminating positions to recover more unique coverage for paralog-limited genes.
  - **Pilot-capture coverage feedback loop** — feed empirical pilot-capture depth back to re-balance bait copy numbers (boost under-covered, trim over-covered).

---

16 Reproducibility and provenance

Every run records what produced it:

- **`provenance.json`** — tool versions captured at runtime, reference releases, the `git_commit`, and the `design_genome` + `specificity_reference` paths (so the no-alt invariant is documented per run).
- **`refs.provenance.json`** (written by `bin/fetch_references.sh`) — source URL, release, and **md5** of each reference.
- **Pinned environments** — `env/captureforge.yml` pins every tool version; the same file builds the Docker and Apptainer images.
- **MANIFEST** — md5 of each deliverable.

To cite the exact references, read the release + md5 fields from `refs.provenance.json` (for a human run, for example: Ensembl r112 GTF, UCSC hg38 rmsk/simpleRepeat, Ensembl Variation REST for rsIDs).

---

17 Glossary

- **Bait / probe** — biotinylated oligo complementary to a target region.
- **Panel** — the full set of baits.
- **Design class** — `coding`/`promoter`/`regulatory`/`cnv`/`str`/`anchor`; drives tiling and masking.
- **Tiling** — laying overlapping baits across a region; "2×" ≈ each base covered by ~2 baits.
- **Masking** — removing repeats/segdups/low-complexity/STR cores so you don't bait non-unique sequence.
- **Specificity** — rejecting baits that also hybridize off-target.
- **Mappable coverage** — covered ÷ tileable target. **Raw coverage** — covered ÷ nominal target.
- **AS-BUILT coverage** — what the final baits actually cover (`final_covered_targets.bed`).
- **No-alt primary assembly** — reference without alternate haplotypes/decoys; required for specificity.
- **Anchor** — an rsID (± flank) you want ≥2 baits to span.

18 FAQ

- **Do I need MANE?** Only for human, and only to prefer the clinical canonical transcript. Other organisms fall back to Ensembl-canonical or the longest transcript.
- **Why is my raw coverage "only" ~67%?** (e.g. in a repeat-heavy panel where the gap is large.) Because a large share of that gap is sequence intentionally masked as uncapturable. Read the **mappable** number (covered ÷ tileable target) instead.
- **Can I run without RepeatMasker/dbSNP/regulatory tracks?** Yes — masking falls back to dustmasker, anchors need a user BED, and `regulatory` classes produce nothing (promoter windows still come from the GTF TSS).
- **Why did a gene come out at 0%?** Check whether the off-targets are alt-haplotype contigs (reference artifact — fix the index) or a real paralog (genuine, partial coverage).
- **Does boosting increase my cost/tier?** No — copies raise representation, not the number of distinct oligos.

19 Citation

Author: Bright Adu, Genomics and Bioinformatics Core Facility, Noguchi Memorial Institute for Medical Research, University of Ghana, Legon, Ghana.

See `CITATION.cff` in the repository root for the machine-readable citation. Cite CaptureForge by name and version, and cite the exact reference releases from `refs.provenance.json`.

20 Version / changelog

- **0.1.0** — initial release: 10-stage Nextflow pipeline, three profiles, three input on-ramps, no-alt reference invariant enforced, QC dashboard + figures, generic order package.